ENME 441

02 – Python Programming

What We'll Cover

Seven parts, spread over several lectures — the whole language, in the order you'll need it.

PART 1

basics

Types, operators, strings

- Numbers, strings, conversion
- Operators
- Formatting & f-strings

PART 2

flow

Deciding and repeating

- if / elif / else
- for, while, range()
- break, continue, iterables

PART 3

functions

Compartmentalizing code

- def, arguments, returns
- lambda expressions
- Iterators & generators

PART 4

collections

Holding many values

- Lists and tuples
- Sets and dictionaries
- Comprehensions

PART 5

files & modules

Working with files and extending the language

- Exception handling
- import and the stdlib
- Reading & writing files

PART 6

bitwise

Comparing and manipulating bits

- Number systems
- Bitwise operators
- Masks and bit fields

PART 7

objects

Structuring larger code

- Classes and instances
- Inheritance, composition
- Dunder methods

Before We Start

1

We will cover the highlights of the standard Python language (Cpython)

Enough of the language to write everything this course asks for. Depth comes from using it, not from watching slides.

2

Full language notes live online

https://go.umd.edu/441 — All Python code covered in class is available here, with more details and examples.

3

Need a structured refresher or starter course? 30-Days-Of-Python

https://github.com/Asabeneh/30-Days-Of-Python is a good option if you are learning Python from scratch or want a refresher with exercises.

4

MicroPython is a subset of what follows

MicroPython is a derivative of Python that runs on microcontrollers. Most of the language transfers directly.

01
Basics

Types, operators, strings

What Makes Python Python

Interpreted

No compile step. Run the source file directly and the interpreter executes it line by line.

Dynamically typed

A variable's type is decided by whatever you last assigned to it.

Strongly typed

Mixing incompatible types raises an error.

Easy to read

Indentation is the block structure, so correct formatting is required.

Automatic memory

No malloc, no free, no delete, no garbage collection.

Exception handling

Errors are catchable objects, so a program can respond to failure instead of stopping.

Everything is an object

Numbers, strings, functions, even modules all carry attributes and methods.

Highly portable

The same source runs on Windows, macOS, Linux, Raspberry Pi, and microcontrollers.

Python Is Easier to Use Than C++

C++

```cpp
// C++ hello world

#include <iostream>
using namespace std;

int main () {
  cout << "Hello world" << endl;
  return 0;
}
```

PYTHON

```python
# Python hello world

print('Hello world')
```

C++

```cpp
// C++ while loop

int x = 0;
while (x < 5) {
    x++;
    cout << x;
}
```

PYTHON

```python
# Python while loop

x = 0
while (x < 5):
    x += 1
    print(x)
```

Core Python Data Types: `int`, `float`, `complex`, `string`

```
size = 40           # integer
f1   = 5.6          # float
z    = 3+1j         # complex

# Complex-number helpers:
abs(z)              # magnitude
z.real              # real part
z.imag              # imaginary part

# Strings — single or double quotes, your choice:
'hello world'
"bye world"
'it\'s hot'         # escape the quote...
"that's nice"       # ...or just use the other one
```

- **int**

  Arbitrary precision – size keeps growing as needed.

- **float**

  Same as a C double (at least 53 bits).
  There is no separate single-precision type.

- **No signed, unsigned**

  Python has one integer type and one float type. The C zoo of width and sign qualifiers is gone.

- **complex**

  Built into the language (like Matlab), written with a j suffix.

Type Conversion

```
# Convert x to an integer:
int(x)

# Convert string s, written in the given base, to an integer:
int(s, base)    # try: int('101',2)  or  int('A',16)

# Convert x to a floating-point value:
float(x)

# Build a complex number from real & imaginary parts:
complex(real, imag)

# Convert x to a string:
str(x)
```

- **Conversion functions return, they don't modify**

  int(x) hands back a new value. x itself is untouched — assign the result if you want to keep it.

- **int() takes a base argument**

  Second argument is the base the string is written in, allowing binary, octal, or hexadecimal to be read from text.

- **Conversion can fail**

  int('cat') raises ValueError (strong typing)

- **Anything can be converted to text via str()**

  Any object can be turned into text; classes control how via __str__ (Part 7).

Comments

```
# this is a single-line comment

"""
This is a multi-line
comment (placed
between triple full quotes)
"""

def fib(n):
    """Print the Fibonacci series up to n.

    n -- upper bound on the series
    Returns nothing; prints as it goes.
    """
```

- **Explain the intent**

  Comment lines and blocks whose purpose isn't obvious from a quick glance.

- **Comment every function**

  What it does, what arguments it takes, and what it returns. A triple-quoted string at the top of the function is the conventional place (this becomes the function's `docstring`)

- **Watch out for smart quotes**

  Copy-pasting from a document or slide can bring curly quotes with it, which Python will not accept.

Operators

Arithmetic operators yield numeric values:

ARITHMETIC	
+	Addition
−	Subtraction
*	Multiplication
/	Division (always a float)
//	Floor division: 20 // 3 → 6
**	Exponent: 2 ** 3 → 8
%	Modulo (remainder): 8 % 3 → 2

Comparison operators yield Boolean values:

COMPARISON	
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Note that / always returns a float, even for two integers with no remainder. Use // when you want the integer part.

Assignment Operators

Every arithmetic operator has a compound assignment form that updates a variable in place.

OPERATOR	EXAMPLE	SAME AS
`=`	`x = 2`	`basic assignment`
`+=`	`x += 2`	`x = x + 2`
`-=`	`x -= 2`	`x = x - 2`
`*=`	`x *= 2`	`x = x * 2`
`/=`	`x /= 2`	`x = x / 2`
`//=`	`x //= 2`	`x = x // 2`
`**=`	`x **= 2`	`x = x ** 2`
`%=`	`x %= 2`	`x = x % 2`

There is no ++ or -- in Python. Use x += 1.

String Operations

Strings are sequences of characters that can be indexed, sliced, and measured

```
# Concatenate with +
word = 'ab' + 'cd'      # 'abcd'

# Index and slice (counting starts at zero):
'Hello'[0]              # 'H'
'Hello'[1:3]            # 'el'   (slice)
'Hello'[2:4]            # 'll'   (slice)
'Hello'[-1]             # 'o'    (last character)

# Length of any iterable, not just strings:
len('abcde')            # 5
```

- **Indices start at zero**

  The first character is [0]. Negative indices count back from the end, so [-1] is the last one.

- **A slice [a:b] stops before b**

  It includes index a and excludes index b, so its length is b - a. Omit either end to run to the start or finish.

- **Strings are immutable**

  Once created, a string cannot be changed. 'Hello'[0] = 'J' is an error — build a new string instead.

- **len() works on any iterable**

  Same function for lists, tuples, sets, dicts and files.

String Methods

Called a method using dot notation: `object.method`

```
s.lower()                   # 'ABC'  -> 'abc'
s.upper()                   # 'abc'  -> 'ABC'
s.capitalize()              # 'abc'  -> 'Abc'
s.strip()                   # remove leading/trailing whitespace

s.isalpha()                 # True if all characters are letters
s.isdigit()                 # True if all characters are digits
s.isspace()                 # True if all characters are whitespace

s.startswith(string_to_check)
s.endswith(string_to_check)
s.count(string_to_count)

s.find(string_to_find)      # index of first occurrence
s.rfind(string_to_find)     # index of last occurrence
s.replace(old_string, new_string)

s.split(delimiter)          # string -> list of pieces
delimiter.join(iterable)    # list of pieces -> string
```

- **(and many more)**

  dir(str) in an interpreter lists all methods
  help(str.split) explains the method details.

- **No methods mutate s**

  Strings are immutable, so each method returns a new string.

- **find() returns -1 on failure**

  It does not raise. Test the result before using it as an index.

- **split() and join() are inverses**

  split() breaks a string into a list; join() glues a list back into a string using the string it's called on as the delimeter.

String Formatting

Three ways to build a string out of values.

Given x = 2.5, y = 'half of', z = 5:

```
# Concatenation: no control over appearance
s = str(x) + ' is ' + y + ' ' + str(z)

# C-style % formatting
s = '%2.1f is %s %d' % (x, y, z)

# The str.format() method
s = '{:2.1f} is {:s} {:d}'.format(x, y, z)

# All three produce:
#   '2.5 is half of 5'
```

- **f = float**
- **s = string**
- **d = decimal (integer)**
- **b = binary**
- **e = exponent**
- **x / X = hexadecimal, lower / upper case**

%width.precision — width is the minimum length (padded with leading spaces), precision is the number of digits after the radix (decimal point).

See https://docs.python.org/3/library/string.html for more format options.

Formatted String Literals: f-strings

```
f'5 modulo 3 = {5%3}'

x = 2
y = 5
f'x + y = {x+y}'

x = 2.5
t = 12
f'{x} * cos({t}) = {x*math.cos(t)}'

# Format specifiers work the same way,
# after a colon inside the braces:
f'5 / 3 = {10/3:5.3f}'
```

- **Any expression goes in the braces**

  Not just a variable name — arithmetic, function calls, method calls, indexing are all ok.

- **Evaluated where it is written**

  The values are read when the f-string is built, not when it is displayed or used.

- **Same format specifiers as format()**

  Everything after the colon is the spec you already know: {value:width.precision type}.

- **Preferred formatting option**

  Shorter than .format(), safer than concatenation, and the expression sits where the value appears.

Importing From Modules (Libraries)

Two forms — import the full module, or bring in one name out of it.

```
import theModule
    or
from theModule import theFunction
```

These two blocks do exactly the same thing:

IMPORT THE MODULE

```
import random

random.randint(5, 20)
```

IMPORT THE NAME

```
from random import randint

randint(5, 20)
```

The Python Standard Library

Shipped with the interpreter — no installation needed.

`time`	time.sleep(), time.time()
`datetime`	Calendar dates and clock times
`sys`	Interpreter and command-line arguments
`os.path`	os.path.basename(), os.path.dirname()
`math`	math.exp(), math.log(), math.sin(), …
`cmath`	The same maths, for complex variables
`random`	Random numbers and random choices
`json`	Read and write JSON — you'll use this a lot later

The library is far larger than this, see docs.python.org/3/library
MicroPython ships cut-down versions of many of these under the same names

Custom Modules

Any Python file is importable as a module.

1

Write a normal .py file

Put the functions you want to share in it — for example new_module.py.

2

Put it beside your main code

Same directory is the simple case; Python also searches the paths in sys.path.

3

Import it by filename, without the .py

import new_module (NOT import new_module.py)

4

Importing runs the file

Every top-level statement in the module executes at import time, not just the def statements.

Important External Packages

`numpy`

High-level maths, multi-dimensional matrix operations, and the array type nearly every other scientific package builds on.

`pandas`

Data manipulation and analysis: data structures and operations for numerical tables and time series.

`matplotlib`

2D plotting library, and the engine underneath most higher-level plotting packages.

`django, flask`

High-level frameworks for writing web applications and services.

None of these run on MicroPython. It does ship ulab, which offers a numpy-like subset.

Breaking a Long Line

The preferred way is Python's implied line continuation inside parentheses, brackets and braces.

```
params = urllib.urlencode(
    {'field1': random.randrange(100,1000),
     'field2': random.randrange(100,1000),
     'key': "74W7TQ5E82VFO8GG"} )


# Where there are no brackets, a trailing backslash
# also continues the line:

total = first_value + second_value + \
        third_value
```

- **Break after an open bracket or a comma**

  Inside any (), [] or {}, a newline is just whitespace — no continuation marker needed.

- **Indent the continued lines**

  Line them up under the opening bracket so the structure stays readable.

- **Add parentheses if there are none**

  Wrapping a long expression in an extra pair of parentheses buys you free line breaks.

- **Backslash is the fallback**

  It works, but nothing may follow it on the line — not even a space — which makes it easy to break by accident.

02

Flow control

Branching and loops

Branching: if

A colon opens the block
Indentation defines block extent

SYNTAX

```
if Boolean_condition:
    code_block_when_True
```

EXAMPLE

```
x = float(input("Enter a number greater than 5:"))

if x <= 5:
    print(f"{x:4.2f} is too small.")
    print("Maybe pay more attention next time?")

if x > 5:
    print(f"Your number is {x:4.2f}.")
```

- **The colon starts the block**

  Every compound statement — if, for, while, def, class — ends its header line with a colon.

- **Indentation is the block**

  Consistently indented lines belong to the block. Four spaces is the convention; be consistent and never mix tabs with spaces.

- **input() always returns a string**

  Convert it before doing arithmetic, as float(input(...)) does here.

Branching: if … elif … else

```
x = int(input("Enter #:"))

if x < 0:
    x = 0
    print("Negative value entered, changed to zero")
elif x == 0:
    print("Zero")
elif x == 1:
    print("One")
else:
    print("More than one")
```

- **Tested top to bottom**

  The first condition that is True runs; every later branch is skipped, even if it would also be True.

- **elif chains as far as you like**

  Any number of elif clauses may follow the if.

- **else is optional**

  It catches everything the earlier tests missed. If you omit it and nothing matches, the whole statement does nothing.

There is no switch/case statement in Python

if __name__ == "__main__"

__name__ is a built-in variable that holds "__main__" when the file is run directly, and the module's name when it is imported.

CONTENTS OF foo.py

```python
def main():
  print("foo main() executed")

if __name__ == '__main__':
  main()
else:
  print("foo imported")
```

CONTENTS OF bar.py

```python
import foo

# try toggling this on and off:
foo.main()
```

THE PATTERN TO USE IN YOUR OWN FILES

```python
def your_custom_function():
  # main code goes here, with the function
  # definitions at the top of the file

if __name__ == "__main__":
  your_custom_function()
```

Run foo.py and it prints "foo main() executed". Import it from bar.py and it prints "foo imported" instead — the guarded call does not fire.

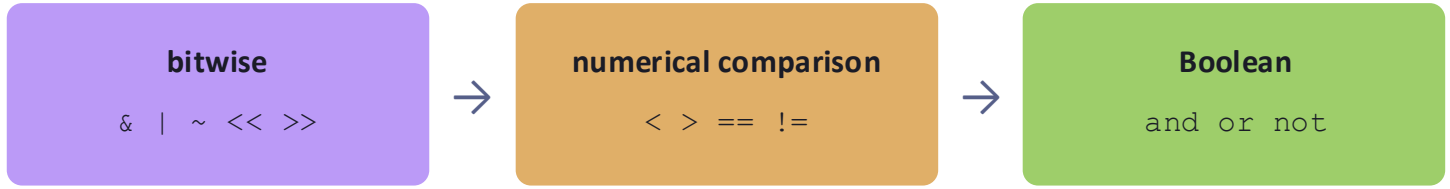
Every file you write that could ever be imported should end this way.

Boolean Operators: and, or, not

Boolean operators compare logical values — True (1) and False (0) — and are written in plain English.

```
if not ((x > 20 and y < 50) or z < 0):
    # do something
```

PRECEDENCE



- **Not the same as & | ~**

  Those are the bitwise operators, covered in Part 6. They operate on the bits of an integer, not on truth values.

- **Precedence bites here**

  1>2 and 4<5 is (1>2) and (4<5) → False. But 1>2 & 4<5 is 1 > (2 & 4) < 5 → 2 & 4 is 0, and 1 > 0 < 5 → True.

- **When in doubt, parenthesize**

  The reader of your code should not have to recall a precedence table.

Flow Control: for

A Python for loop walks the elements of an iterable

SYNTAX

```
for loop_variable in iterable:
    # Do something.
    # Never modify the iterable within the loop!
```

EXAMPLE

```
my_list = ['cat', 'window', 'defenestrate']

for x in my_list:
    print(x, len(x))
```

Python assigns the current element to the loop variable (x above), then runs the loop body once per element, in order.

- **Iterables**

  A data structure that hands its members over one at a time. Lists are the common example.

- **No loop variable**

  loop ends when the iterable runs out.

- **Don't modify the iterable inside**

  Adding or deleting elements while iterating over them gives results you did not intend. Build a new one instead.

Iterables Return Their Members One at a Time

```
for item in [1, 2, 3]:              # list
    print(item)

for item in range(2, 1000):         # range object
    print(item)

for item in (1, 2, 3):              # tuple (immutable list)
    print(item)

for key in {'one':1, 'two':2}:      # dictionary (dict)
    print(key)

for char in "123":                  # string (str)
    print(char)

for line in open("myfile.txt"):     # file (TextIOWrapper)
    print(line, end='')
```

- **One protocol, many types**

  Every iterable provides an __iter__() method. Try help(list) or dir(str) to see it.

- **A dict iterates over its keys**

  Not its values — use .values() or .items() when you want those (Part 4).

- **A file iterates over its lines**

  Including the trailing newline, which is why print() gets end='' here.

- **You can write your own**

  Adding __iter__() to your own class makes it work in a for loop like anything else (Part 7).

for loops and range()

range() produces an iterable of integers

```
range(       stop)
range(start, stop)
range(start, stop, step)
```

start (default = 0)

stop — the loop ends before this value

step (default = 1)

```
for i in range(5):           # [0, 1, 2, 3, 4]
    print(i)

for x in range(2, 15, 3):    # [2, 5, 8, 11, 14]
    print(x)
```

for loops: break, continue

break leaves the loop entirely
continue skips to the next iteration of the inner-most loop

```
for x in range(2,6):            # outer loop, x = 2,3,4,5

    for y in range(1,6):        # nested loop, y = 1,2,3,4,5

        if x == y:
            continue            # go directly to next y value

        # keep going if 'continue' was not called...

        print(f'{x}, {y}')

        if x + y > 5:
            break               # exit inner loop, next x value
```

- **Both act on the innermost loop**

  In this example they affect the y loop only. The x loop keeps going either way.

- **continue: skip the rest of this pass**

  Execution jumps back to the top and takes the next value from the iterable.

- **break: abandon the loop**

  The remaining values are never visited and execution resumes after the loop body.

- **To leave both loops**

  There is no labelled break in Python. Use a flag variable, or put the loops in a function and return.

Looping with Index Values: enumerate()

Print the index of every element greater than 5. In C++ the index is the loop variable — in Python it isn't there at all.

C++ — index is the loop variable

```cpp
for (int i=0; i<theList.size(); i++) {
  if (theList[i] > 5) {
    cout << i;   // print the loop variable
  }
}
```

PYTHON — no index to print

```python
theList = [4, -2, 98, 2.5, 6.02]

for value in theList:
  if value > 5:
    print(???)   # no loop variable!
```

enumerate() wraps an iterable and yields (index, value) pairs:

```python
for (i, value) in enumerate(theList):
  if value > 5:
    print(i)
```

The parentheses around (i, value) are optional — this is tuple unpacking, and you'll see the same pattern again with dictionaries and zip().

Flow Control: while

Repeat as long as a condition is true. Use when the number of iterations is not known in advance.

```
a = -10
while a <= 10:
    print(a)
    a += 1          # same as a = a + 1


while True:
    if int(input('enter number > 10 to end: ')) > 10:
        break


import time
while True:                 # infinite loop
    print('.')
    time.sleep(0.25)        # pause for 1/4 second
```

- **Something must change to exit**

  If nothing in the body can make the condition False, the loop never ends.

- **while True + break**

  The idiomatic way to write "loop until something happens" — the exit test lives where the decision is made.

- **Deliberate infinite loops are normal**

  On a microcontroller the main loop never exits. time.sleep() keeps it from spinning at full speed.

Two Small Rules

pass — syntactic filler

```
while 1:
    pass
```

A block cannot be empty in Python. When you need a block that does nothing — a stub function, an except clause you want to ignore, a placeholder class — pass is the statement that fills it.

No assignment inside a loop expression

✗

```
while (x = f.readline()) != "end":
    # do something
```

✓

```
x = f.readline()
while x != "end":
    x = f.readline()
    # do something
```

Unlike C and C++, an assignment is a statement in Python, not an expression — so it cannot appear inside a condition. Read the value first, then test it.

LAB ASSIGNMENT

Lab 1 — Loops

- Flow control: if / elif / else
- for and while loops, range()
- break and continue
- Formatted output with f-strings

03
functions

Naming a block of code

Defining Custom Functions

def names a block of code so it can be run from anywhere, as many times as you like.

SYNTAX

```
def function_name():
    # function code goes here
    # and ends when the indents stop
```

EXAMPLE

```
# Print a Fibonacci series up to n,
# and display the result:
def fib(n):
    a, b = 0, 1
    while b < n:
        print(b)
        a, b = b, a+b
    print(a)
```

- **Define before you call**

  The def must be executed before the first call. In practice: put your functions at the top of the file.

- **a, b = b, a+b**

  Both right-hand values are computed first, then assigned. No temporary variable needed to swap or advance a pair.

- **The body ends when the indent does**

  There is no closing brace or 'end' keyword — the first line back at the outer indent level is outside the function.

Passing & Returning Values

```
# Find the maximum of two values:
def maximum(x, y):          # pass multiple parameters
    if x > y:
        max = x
    else:
        max = y
    return max              # return the result

# Call the function:
print(maximum(2, 3))        # order matters

print(maximum(y=3, x=2))    # order doesn't matter
```

The same function, written more compactly:

```
def maximum(x, y):
    if x > y:
        return x
    else:
        return y
```

```
def maximum(x, y):
    if x > y: return x
    else: return y
```

- **Positional or keyword**

  maximum(2,3) matches by position. maximum(y=3, x=2) matches by name, so the order stops mattering.

- **return exits immediately**

  Nothing after it in the function runs — which is why the compact versions need no max variable.

- **No return? You get None**

  A function that falls off the end returns None. Assigning that result is a common source of confusion.

Recursive Functions

A function is allowed to call itself.

```
def factorial(n):
    if n == 1:
        return 1                    # base case — stops the
recursion
    else:
        return n * factorial(n-1)   # recursive case


factorial(4)
#   -> 4 * factorial(3)
#      -> 3 * factorial(2)
#         -> 2 * factorial(1)
#            -> 1
```

- **Base case first**

  Without a condition that returns without recursing, the calls never stop.

- **Each call gets its own variables**

  n inside factorial(3) is a different n from the one in factorial(4). Local names do not leak between calls.

- **Python limits the depth**

  Roughly 1000 nested calls by default, then RecursionError. Deep recursion is usually better written as a loop.

Default Argument Values

```
def maximum(x, y=5):
    if x > y: return x
    else: return y


print(maximum(1))       # returns 5   (y defaults to 5)

print(maximum(6))       # returns 6

print(maximum(1, 2))    # returns 2   (default overridden)
```

- **Defaults go at the end**

  Every parameter with a default must come after all the parameters without one — otherwise the call is ambiguous.

- **Lets one function serve several uses**

  The common case is short to call; the unusual case is still reachable.

- **Evaluated once, at definition**

  A mutable default (a list, a dict) is shared by every call. Use None as the default and build a fresh one inside.

Arbitrary Number of Arguments

Prefix an argument with * and multiple passed values are packed into a single tuple.

ALL ARGUMENTS COLLECTED

```
def my_sum(*vals):
    sum = 0
    for v in vals:
        sum += v
    return sum


my_sum(1, 2, 3, 4)    # 10
```

COMBINED WITH A NORMAL ARGUMENT

```
def sum_prod(m, *vals):
    sum = 0
    for v in vals:
        sum += v
    return m * sum


sum_prod(2, 1, 2, 3)   # 12
```

1 **vals is an ordinary tuple**

Loop it, index it, len() it — inside the function it is just a tuple of whatever was passed.

2 **The starred parameter goes last**

Normal parameters are filled first; * sweeps up everything left over. print() and max() work exactly this way.

Lambda Functions

A lambda is a small single-expression function. Written inline, it needs no name at all.

```
lambda bound_variables: function_body
```

REGULAR FUNCTION

```
def add(x, y):
    return x + y


sum = add(1, 2)
```

EQUIVALENT LAMBDA

```
add = lambda x, y: x + y



sum = add(1, 2)
```

1

One expression, and its value is returned

There is no return keyword and no room for statements — if you need a block, write a def.

2

The point is being anonymous

Assigning a lambda to a name, as above, is legal but pointless. Lambdas earn their keep passed straight into another function .

Lambda Function Use

(1) As an argument

```
colors = ['blue', 'red', 'green', 'orange']
colors.sort(key=lambda x: len(x))

vals = [2, -3, -4, 6, 3]
vals.sort(key=lambda x: x*x)
```

sort() calls the key function once per element and sorts on the results

(2) In a return statement

```
def func(x):
    return lambda a: x**a

pow2 = func(2)      # pow2(n) = 2**n
pow10 = func(10)    # pow10(n) = 10**n
```

A function that builds and returns another function. The returned lambda remembers the x it was created with.

Iterators via iter()

An iterator is an iterable you step through by hand, one next() call at a time.

```
mylist = [5, 1, 0, 6, 7]

it = iter(mylist)    # declare an iterator

print("Here is the first value in the list:")
print(next(it))

print("Now the second value:")
print(next(it))

print("And the sum of the rest:")
print(next(it) + next(it) + next(it))
```

- **iter() takes an iterable, returns an iterator**

  The iterator holds the position; the original list is unchanged.

- **next() advances it**

  Each call hands back the next value and moves the position forward one.

- **It runs out**

  Calling next() past the last value raises StopIteration — which is exactly how a for loop knows to stop.

- **No rewind**

  An iterator only moves forward. To start over, call iter() again on the original iterable.

Iterators via Generators

A generator is a function that behaves like an iterator: yield hands back a value and freezes the function where it stands, and a next() call steps through the next iteration cycle.

```
# Define a generator function:
def fibonacci():
    n1 = 0
    n2 = 1
    while True:
        yield n1          # value returned for this iteration
        n1, n2 = n2, n1 + n2

f = fibonacci()           # declare an iterator

# Print the first 10 values in the sequence:
for _ in range(10):
    print(next(f))
```

- **yield, not return**

  return ends a function for good. yield suspends it — the next next() picks up on the following line.

- **Local state survives**

  n1 and n2 keep their values between calls, so the function remembers where the sequence had got to.

- **An infinite sequence is fine**

  while True never ends, but nothing is computed until asked for. Only the values you take are ever produced.

- **Also works directly in a for loop**

  for x in fibonacci(): — though with no break this one runs forever.

Generators: multiple yield statements

A generator may contain multiple yield statements rather than a single yield within a loop.

```
def gen(x):
    yield x**2
    yield x**3
    yield x**4

g = gen(2)          # declare a generator

for _ in range(3):
    print(next(g))  # 4, then 8, then 16

next(g)             # StopIteration — nothing left to yield
```

- **Executed in order**

  Each next() runs to the following yield, so the values arrive in the order they are written.

- **Running past the end is an error**

  Calling next() after the last yield raises StopIteration. A for loop handles that for you; manual next() calls do not.

- **Generators cannot be reset**

  There is no rewind. Call the generator function again to get a fresh one starting from the top.

Global vs. Local Variables

A function can read a global variable freely. Changing one takes an explicit declaration.

READING A GLOBAL — NOTHING SPECIAL NEEDED

```
c = 5           # global variable

def add1(x):
    return c + x

c = add1(3)     # c is now 8
```

WRITING A GLOBAL — DECLARE IT

```
c = 5           # global variable

def add2(x):
    global c
    c += x

add2(3)         # no assignment needed
```

1 **Assignment inside a function creates a local**

Without the global keyword, c += x would make a brand-new local c — and fail, because it has no value yet.

2 **Prefer arguments and return values**

Globals make a function's behaviour depend on things the call site cannot see. Pass what you need in, hand the result back out.

04 collections

Lists, tuples, sets, dictionaries

Lists, Tuples, Sets, and Dictionaries

Beyond strings, four iterable data structures carry nearly all the data you'll handle. All four hold heterogeneous types.

List

Mutable and **ordered**. A sequence whose elements can be changed, added to, and removed.

Written with [ ].

Tuple

Immutable and **ordered**. Fixed once created — and faster than a list because of it.

Written with ( ).

Set

Mutable and **unordered**, with no duplicates. Built for membership tests.

Written with { }.

Dictionary

Mutable and **ordered** pairs of (key, value). Look a value up by its key.

Written with { key: value }.

Notes:

- *Dictionaries in Python versions before 3.7 were unordered.*
- *You can build your own iterable types by giving a custom class an __iter__() method*

Lists

Mutable and ordered: elements can change, and each one keeps its position.

```
a = ['foo', 'bar', 100, 123.4, 2*2]

# Indexing and slicing — same rules as strings:
a[0]        # 'foo'
a[1:2]      # ['bar']
a[1:3]      # ['bar', 100]
a[:2]       # ['foo', 'bar']
a[2:]       # [100, 123.4, 4]

# Lists can contain lists (or any other structure):
myList = [1, 2, [3, 4, 5]]    # 3rd element is a list
myList[2][1]                  # 4

# Assignment — this is what "mutable" means:
a[2] += 23        # change one element
a[0:2] = [1,12]   # reassign a slice
a[0:2] = []       # delete elements 0 and 1

len(a)            # 3 after the deletion above
a = 5*[None]      # empty list of 5 elements
```

- **A slice returns a list**

  Even a one-element slice: a[1:2] is ['bar'], while a[1] is 'bar' itself.

- **Elements need not match**

  Strings, ints, floats and other lists can sit side by side in one list.

- **Assigning to a slice resizes**

  The replacement need not be the same length — assigning [] to a slice deletes it.

- **n*[None] preallocates**

  Handy when you need a fixed-length list to fill in by index.

List Methods

Called on the list with a dot. Unlike string methods, most of these change the list in place.

```
l.append(x)        # append x to the end of the list
l.extend(L)        # append all items in list L
l.insert(i, x)     # insert x at index i

l.remove(x)        # remove the first item with value x
l.pop(i)           # remove and return the ith value
l.pop()            # remove and return the last value

l.index(x)         # index of the first value x
l.count(x)         # how many times x appears

l.reverse()        # reverse the list in place
l.sort(key=None)   # sort in place, with or without a key
```

- **These modify l, and return None**

  l = l.sort() throws your list away. Call l.sort() on its own line.

- **append vs extend**

  append() adds one element — even if that element is a list. extend() adds each item of its argument separately.

- **sorted() is the copy version**

  sorted(l) returns a new sorted list and leaves l alone. It works on tuples and any other iterable too.

- **key= takes a function**

  The lambda from Part 3 goes here: l.sort(key=lambda x: len(x)).

Removing List Items: del, pop, remove

Three ways, distinguished by what you know about the item — its position, or its value.

```
a = ['foo', 'bar', 100, 1234, 2*2, 5, 'foobar', -1]
```

del

by position, discard it

```
del a[0]      # remove 1st element
              # = a[0:1] = []

del a[2:4]    # remove slice 2 & 3
              # = a[2:4] = []
```

pop()

by position, keep the value

```
val = a.pop()     # remove the last
                  # element, into val

val = a.pop(2)    # remove the 3rd
                  # element, into val
```

remove()

by value, first match only

```
a.remove('bar')   # remove the first
                  # element with the
                  # given value
```

1 **del is a statement, not a method**

It works on slices and on whole names — del a removes the variable itself.

2 **remove() raises if the value isn't there**

ValueError. Test with in first if you are not sure.

Checking Membership: in

The same keyword that drives a for loop also asks a yes/no question.

Walking an iterable — the use you have already seen:

```
for i in range(10):
    print(i)
```

Testing membership — works on any iterable:

```
the_list = ["this", "that", "those", "these", "them"]
if "this" in the_list:
    print("the list contains 'this'")


the_string = 'abracadabra'
if 'c' in the_string:
    print(f"c is at position {the_string.find('c')}")
```

For a string, in tests for a substring, not just a single character. not in is the negation, and reads better than not (x in y).

List Comprehension

Compact syntax for building a list from an iterable — and significantly faster than the equivalent loop.

```
[expression for item in iterable]
```

STANDARD for LOOP

```
x = [1, 3, 5, 2]
y = []

for val in x:
    y.append(val**3)
```

LIST COMPREHENSION

```
x = [1, 3, 5, 2]

y = [val**3 for val in x]
```

1

Read it left to right

The expression on the left is what goes in the new list; the for on the right says where the values come from.

2

It builds a new list

The source iterable is untouched — which is why comprehensions sidestep the "never modify the iterable" rule.

List Comprehension: Embedded for Loops

More than one for clause may appear — they nest, left to right, exactly as written-out loops would.

```
v1 = [2, 4, 8]
v2 = [4, 3, -9]
```

One iterable

```
w = [val**val for val in v1]
```

→ [4, 256, 16777216]

Element-wise (dot product terms)

```
[v1[i]*v2[i] for i in range(len(v1))]
```

→ [8, 12, -72] — index into both lists at once

Every pair (cross product)

```
[x*y for x in v1 for y in v2]
```

→ 9 values: the second for runs fully for each x

*The leftmost for is the outer loop. [x*y for x in v1 for y in v2] is the same as nesting a for y inside a for x.*

List Comprehension: Embedded Conditionals

A trailing if filters which values make it into the result.

Given v1 = [2, 4, 8] and v2 = [4, 3, -9]:

```
z1 = [3*x for x in v1 if x > 3]
```

```
x = 2  !> 3                z1 = []
x = 4  >  3    3*4 = 12    z1 = [12]
x = 8  >  3    3*8 = 24    z1 = [12, 24]
```

`if` goes at the end

It is tested after the for clauses have produced a candidate value, and it decides whether that value is kept.

```
z2 = [x*y for x in v1 for y in v2 if x*y > 0]
```

```
x*y = 2*4  =  8 > 0    z2 = [8]
x*y = 2*3  =  6 > 0    z2 = [8, 6]
x*y = 2*-9   !> 0      z2 = [8, 6]
x*y = 4*4  = 16 > 0    z2 = [8, 6, 16]
x*y = 4*3  = 12 > 0    z2 = [8, 6, 16, 12]
x*y = 4*-9   !> 0      z2 = [8, 6, 16, 12]
x*y = 8*4  = 32 > 0    z2 = [8, 6, 16, 12, 32]
x*y = 8*3  = 24 > 0    z2 = [8, 6, 16, 12, 32, 24]
x*y = 8*-9   !> 0      z2 = [8, 6, 16, 12, 32, 24]
```

Copying Lists: Deep vs. Shallow Copies

The assignment operator does not copy a list — it binds a second name to the same object.

SHALLOW — TWO NAMES, ONE LIST

```
a = [2, 4, 6]
b = a
print(a)          # [2, 4, 6]
b[1] = -1
print(a)          # [2, -1, 6]   (changed!)

id(a) == id(b)   # True
```

DEEP — SLICE TO GET A REAL COPY

```
a = [2, 4, 6]
b = a[:]          # slice for a deep copy
print(a)          # [2, 4, 6]
b[1] = -1
print(a)          # [2, 4, 6]   (unchanged)

id(a) == id(b)   # False
```

Slicing is only one level deep. If the list contains lists — or any other nested objects — those inner objects are still shared. For nested structures, use deepcopy():

```
from copy import deepcopy
b = deepcopy(a)
```

id() returns an object's identity — in CPython, its memory address. It is the direct way to ask whether two names refer to the same object.

LAB ASSIGNMENT

Lab 2 — Functions, Generators, and Lists

- Defining and calling your own functions
- Generators and the yield statement
- List construction, indexing and methods
- List comprehension

Tuples

Like lists, but immutable — and faster as a result. Define one with values separated by commas.

```
t = (123, 543, 'bar')    # parentheses not required
print(t)                 # (123, 543, 'bar')
print(t[0])              # 123

# Tuples may be nested:
u = t, (1, 2)
print(u)                 # ((123, 543, 'bar'), (1, 2))

# Methods — only two, because nothing can change:
t.count(x)               # occurrences of x in the tuple
t.index(x)               # smallest index of x in the tuple
```

- **The commas make it a tuple**

  Not the parentheses. t = 1, 2, 3 is a tuple; the brackets are there for readability and to disambiguate.

- **Indexed and sliced like a list**

  Everything that reads a list works. Everything that writes one does not.

- **Think C struct**

  A fixed group of related values passed around as one thing — which is exactly what enumerate() and dict.items() hand you.

- **A one-element tuple needs a comma**

  (5) is just the number 5 in parentheses. (5,) is a tuple.

Other Iterable Functions

Built-in functions that work across every iterable type — not methods, so they wrap the iterable rather than belonging to it.

`any()`	True if any element is true	`all()`	True if all elements are true
`len()`	Number of items in the object	`max()`	Largest element
`min()`	Smallest element	`sum()`	Adds the items of an iterable
`sorted()`	Returns a sorted list from any iterable	`reversed()`	Returns a reversed iterator
`map()`	Applies a function to every element	`filter()`	Iterator of the elements that are true
`zip()`	Iterator of tuples, pairing up several iterables	`enumerate()`	Like zip(), but pairing with sequential indices
`iter()`	Returns an iterator for an object	`slice()`	Creates a slice object, as range() does for ranges
`tuple()`	Builds a tuple from any iterable	`list()`	Builds a list from any iterable

Sets

An unordered collection of unique elements — built for membership testing and removing duplicates.

```
a = set('abracadabra')
# {'a', 'r', 'b', 'c', 'd'}   — duplicates gone

b = {5, 10-9, 10<9}
# {5, 1, False}               — braces also make a set

# Sets support the mathematical operations:
a | b    # union
a & b    # intersection
a - b    # difference
a ^ b    # symmetric difference
```

- **No duplicates, ever**

  Adding a value that is already present does nothing. That property is most of what sets are for.

- **No order, so no indexing**

  There is no a[0]. Sets answer "is it in here?", not "what is at position 3?".

- **Empty set is set(), not {}**

  Bare braces make an empty dictionary — the braces were a dict's first.

- **Membership is fast**

  x in a_set does not scan the whole collection the way x in a_list does.

Set / List Conversion

Because a set drops duplicates, a round trip through one is the standard way to deduplicate a list.

```
a = 'abracadabra'
```

a string is just a sequence of characters

↓

```
a = set(a)
```

{'a', 'r', 'b', 'c', 'd'} — duplicates removed

↓

```
a = list(a)
```

['a', 'r', 'b', 'c', 'd'] — back to an indexable, ordered list

The order of a set is not the order you put things in, so the resulting list order is not meaningful. Sort it if the order matters.

Dictionaries

A set of key : value pairs. Look a value up by its key instead of by its position.

1

A key is a string or an integer

Anything immutable, in fact. The value can be any type at all — including another dictionary.

2

Keys are unique

Assigning to a key that already exists replaces its value rather than adding a second entry.

3

Order does not matter

You reach an element by name, not by index, so there is no first or last to speak of.

4

Easy to access, delete, or replace

Everything is done through the key: foo['count'], del foo['count'], foo['count'] = 5.

Dictionary Example

Start empty and fill it in, or write the whole thing at once.

```
foo = {}                        # empty dictionary

foo["word"] = "hi there"        # add a string
foo["count"] = 99               # add an integer
foo["list"] = ["bar", 25]       # add a list
foo[4] = "foobar"               # use an integer as a key

print(foo)
print(foo["word"])              # 'hi there'
print(foo["list"][1])           # 25
print(foo[4])                   # 'foobar'

# Or construct it directly:
foo2 = {"foobar":123, "this":"that", 2:(123, 'x')}
```

- **Assignment creates the entry**

  There is no "add" step — writing to a key that isn't there yet makes it.

- **Values can be any type**

  Strings, numbers, lists, tuples, other dictionaries. Index straight through them: foo["list"][1].

- **Keys can be mixed**

  Strings and integers can key the same dictionary, as "word" and 4 do here.

- **A missing key raises KeyError**

  Use key in foo to check first, or foo.get(key) to get None instead of an error.

Manipulating Dictionaries

Changing entries, and the three methods that turn a dictionary back into something you can loop over.

```
del foo["count"]           # delete the given entry
foo["word"] = 26           # change the value (and its type!)
foo["new word"] = "yo"     # add a new entry
```

`d.keys()`	dict_keys — a view of every key
`d.values()`	dict_values — a view of every value
`d.items()`	dict_items — an iterator of (key, value) tuples

1. **Wrap them in list() to get a list**

   list(d.keys()) gives an ordinary list, so every list method becomes available on the dictionary's data.

2. **= does not copy a dictionary either**

   Same reference-binding behaviour as lists. Use copy.deepcopy() when you need an independent one.

Iterating on a Dictionary

Keys, values, or both at once — plus zip(), which does the same job across two separate lists.

KEYS, THEN LOOK UP THE VALUE

```
q_and_a = {
    'name': 'Lancelot',
    'quest': 'to seek the holy grail',
    'favorite color': 'blue'}

for q in q_and_a.keys():
    a = q_and_a[q]
    print(f'What is your {q}? It is {a}.')
```

BOTH AT ONCE WITH items()

```
knights = {
  'gallahad': 'the pure',
  'robin': 'the brave' }

for (key, value) in knights.items():
  print(key, value)
```

VALUES ONLY

```
print(f'Answers:')
for a in q_and_a.values():
    print(a)
```

TWO LISTS IN STEP WITH zip()

```
questions = ['name', 'quest', 'favorite color']
answers = ['lancelot', 'to seek the grail', 'blue']

for q, a in zip(questions, answers):
    print(f'What is your {q}? It is {a}.')
```

Iterating a dictionary directly — for q in q_and_a: — yields the keys, same as .keys(). You cannot go the other way: values do not identify their keys.

05

Exception handling

Exception Handling: try, except

When an error occurs the interpreter throws an exception and the program stops. Catch it and you decide what happens instead.

```
while 1:
    try:
        x = int(input("Enter a number: "))

    except ValueError:
        print("Not a valid number")

    # code continues here, even if there was an exception
```

- **1 — run the try block**

  Everything inside it is attempted normally.

- **2 — no exception? skip the except**

  Execution carries on after the whole statement.

- **3 — exception? run the except block**

  Then carry on after the statement — the program does not stop.

- **Name the exception you expect**

  except ValueError catches a bad conversion and nothing else. Catching everything hides the bugs you did not anticipate.

Exception Handling: else, finally

Two more clauses: one for the no-error path, one that always runs.

```
import math

while 1:
    try:
        x = input("Enter a number >= 0: ")
        y = math.sqrt(float(x))

    except ValueError:    # catches both <0 and non-numbers
        print("Not a valid number")

    else:                 # only if no exception was raised
        print("Still doing stuff")

    finally:              # always, exception or not
        print("Go again")
```

- **else — the clean path**

  Runs only when the try block finished without an exception. Keeps the "it worked" code out of the try block, so it can't accidentally be caught.

- **finally — no matter what**

  Runs whether or not an exception was raised, whether or not it was caught, and even if the try block returns out of a function. This is where cleanup goes.

Exception Handling: A Real Example

File I/O is the classic case — the file may simply not be there, and that is not a bug in your code.

```
filename = 'data.txt'

try:
    f = open(filename, 'r')

except IOError:
    print(f'cannot open {filename}')

else:
    for (linenum, linetext) in enumerate(f):
        if 'my text' in linetext:
            print(f'found in line {linenum}')
    f.close()

finally:
    print('All done')
```

- **Only the risky line is in the try**

  open() is what can fail. Keeping the try block small means the except clause can only be triggered by the thing you meant.

- **The work goes in else**

  If the file opened, read it. Nothing here can be mistaken for an open failure.

- **Cleanup goes in finally**

  Which is exactly the job the with statement will do for you a few slides from now.

Seeing What Actually Went Wrong

Catching an exception object lets you print it — which is how you debug the failure you just caught.

```
try:
    print(1/0)

except Exception as e:
    print(e)            # division by zero
    print(e.__doc__)    # Second argument to a division or modulo
                        # operation was zero.
```

SOME BUILT-IN EXCEPTIONS

ZeroDivisionError	FloatingPointError
EOFError	KeyboardInterrupt
FileExistsError	FileNotFoundError

Full list: docs.python.org/3/library/exceptions.html

- **as e binds the exception object**

  e carries the message, the type, and the documentation string for that error class.

- **Most errors subclass Exception**

  So except Exception catches nearly everything — but not quite everything.

- **A bare except: catches literally all**

  Including KeyboardInterrupt, so ctrl-C can no longer stop your program. Avoid it.

- **You can define your own**

  Subclass Exception (or BaseException) to raise errors that mean something in your problem domain.

06 Files

File I/O

Reading or writing a file starts by creating a file object.

```
f = open("myfile.txt")


line = f.read()       # read from the current position to the
                      # end, into a string, newlines included

f.write('new text')   # add a string to the file at the
                      # current position

f.close()             # close the file

f.seek(n)             # set the file position
                      #   n = byte offset (1 byte per character)
                      #   n = 0 is the start of the file
```

- **f is an object**

  The methods below belong to it. Everything you do to the file goes through f.

- **There is a current position**

  Reads and writes happen there, and advance it. seek() is how you move it back.

- **read() with no argument reads it all**

  For a large file that is a lot of memory — iterate over lines instead.

- **Always close what you open**

  Buffered writes are not guaranteed to reach the disk until you do.

File I/O: Modes and Encoding

The second argument to open() says what you intend to do with the file.

`f = open(filename, 'r')`	read (the default — 'r' can be omitted); the file must exist
`f = open(filename, 'w')`	write, overwriting the file; creates it if there is none
`f = open(filename, 'a')`	write, appending to the end; creates it if there is none
`f = open(filename, 'x')`	write a new file only; error if the file already exists
`f = open(filename, 'r+')`	read + write; the file must exist, content preserved
`f = open(filename, 'w+')`	read + write, overwriting; creates it if there is none

Text encoding is platform-dependent — cp1252 on Windows, utf-8 nearly everywhere else. Say which one you mean and your code stops depending on where it runs:

```
f = open("myfile.txt", mode = 'r', encoding = 'utf-8')
```

```
f.close()
```

File Objects as Iterators

A file object behaves like a list of strings, one per line — and iterating it is how you read a large file.

```
f = open("data.txt", 'r')

for line in f:
    print(line.strip())    # strip() removes the newline

f.close()


# A file can only be traversed once. To go through it
# again, either close it and open a new file object,
# or rewind the position:

f.seek(0)
```

- **Each line includes its newline**

  Which is why strip() shows up in nearly every loop like this one.

- **Only one line is in memory at a time**

  Unlike f.read(), this works on a file far larger than your RAM.

- **The iterator is one-way**

  Once exhausted, the loop simply does not run again — no error, just nothing. seek(0) resets it.

Accessing Files using with

The with statement handles closing the file for you — including when something goes wrong.

```
with open("test.txt", mode = 'r+') as f:
    f.write("hello there")
    # f.close() is not needed
```

MicroPython

with open statements are not (yet) supported. On the ESP32 you close files yourself.

1

The file closes at the end of the block

As soon as the indented block finishes, whichever way it finishes.

2

Including when an exception is raised

This is the real reason to use it — it replaces the try / finally you would otherwise have to write by hand.

3

as f names the file object

Inside the block, f is exactly the same kind of object open() returns.

Example 1

Write Python code to do the following:

1. **Query the user for a number**
2. **Compare it to every value in a file called data.txt**
3. **Display all numbers in the file larger than the user's number**

You may assume data.txt exists, and that it holds a single real number on each line.

Try it

Create a new Python code file · create a data.txt with one number per line · write and test your code.

Example 2

data.txt contains a list of numbers, one per line. Calculate the sum of every value in the file.

STRAIGHTFORWARD

```
total = 0

with open("data.txt") as f:
    for val in f:
        total += float(val)
```

MORE PYTHONIC

```
with open("data.txt") as f:
    data = f.read().split('\n')
    total = sum(float(n) for n in data)
```

1

read() returns the whole file as one string

Everything, newlines included.

2

split() breaks it on the separator

Splitting on '\n' gives a list with one element per line.

3

The comprehension does the conversion

float() cannot take a list, so convert element by element on the way into sum().

What Do We Know at This Point?

Everything below is fair game — and enough to write real programs.

The language itself

Variables and scope · type conversion · strings and string methods · Boolean operators · formatted output with f-strings

Flow control

if / elif / else · for loops and range() · while loops · break and continue · pass

Functions and data

Custom functions · lists, tuples, sets and dictionaries with their methods · iteration and enumeration · comprehensions

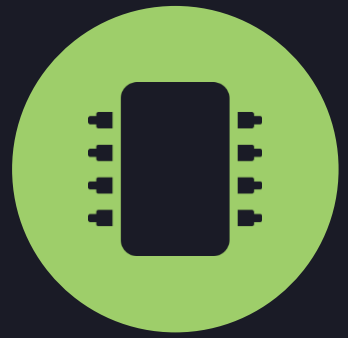
Talking to the outside

Exception handling · importing modules · keyboard input · reading and writing files

06

Bitwise operators

Down at the bit level



Number Systems

Hardware talks in hex and binary. String formatting is how you read and write those bases in Python.

```
print(f'{0x40:02d}')      # display hex as decimal
print(f'{0x40:08b}')      # display hex as binary
print(f'{0b1111:02X}')    # display binary as hex (upper case)
print(f'{0o31:02d}')      # display octal as decimal
```

THE TYPE CHARACTER SELECTS THE BASE

`b`	binary (base 2)
`o`	octal (base 8)
`d`	decimal (base 10)
`x`	hexadecimal (base 16)
`X`	hexadecimal, upper case

- **Literals carry their own prefix**

  0b for binary, 0o for octal, 0x for hex. All four are the same integer once read — the base is only notation.

- **The leading number is the width**

  08b pads to eight characters with zeros. Python does not track a word length, so this is how you show leading zeros.

- **An I2C address is just a number**

  hex() and these format specs are two ways to show the same value in the base the datasheet uses.

Number System Conversion

The powers of two, written four ways. Worth recognising on sight — these are the bit positions in a byte.

DECIMAL	BINARY	OCTAL	HEXADECIMAL
1	00000001	001	01
2	00000010	002	02
4	00000100	004	04
8	00001000	010	08
16	00010000	020	10
32	00100000	040	20
64	01000000	100	40
128	10000000	200	80

Each row has exactly one bit set — which is what makes them useful as masks. Note how one hex digit is exactly four bits, so 8-bit values are always two hex characters.

Bitwise Operators

Bitwise operations manipulate the individual bits inside a binary word — the level hardware actually works at.

Bit
a single binary value (0 or 1)

Nibble
a 4-bit word

Byte
an 8-bit word

1

There is no maximum word length

Python integers have no fixed width — a binary word can grow to any length. Nothing truncates at 8, 16 or 32 bits for you.

2

Why do we care?

Bit fields, bit masks, bit flags, finite state machines, graphics operations, compression, encryption, and every communications protocol you will meet in this course.

3

Hardware registers are bit fields

Configuring a peripheral means setting particular bits of a particular register. That is the whole reason this section exists.

Reference: wiki.python.org/moin/BitwiseOperators · Lecture code: github.com/ddevoe-umd/enme202/blob/main/python/16_bitwise_operators.py

The Bitwise Operators

Six operators, and the compound-assignment forms that go with them.

```
x & y      # AND  (ampersand)
x | y      # OR   (bar)
x ^ y      # XOR  (caret)
~x         # NOT / one's complement (tilde)
x << n     # shift left by n bits
x >> n     # shift right by n bits
```

Combine with assignment, as with arithmetic:

```
x = 1      # binary: 0001
x <<= 2    # binary: 0100
x |= 3     # binary: 0111
```

Declaring and reading binary words:

```
x = 0b0101             # 0b literal
x = int('0101', 2)     # same value, from a string

bin(5)                 # '0b101' — integer to string
print(f"{x:04b}")      # '0101'  — with a fixed width
```

1

<< n multiplies by 2**n

And >> n divides by it, discarding the bits that fall off the right-hand end.

2

Python forgets leading zeros

print(x) for 0b0101 gives 5, and bin(x) gives '0b101'. If you know the word length, format it with a width.

Bit Shifting in Practice

Shifting right discards bits; shifting left does not, because the word simply grows.

```
x = 0b0101
print(f"x =                {x:04b}")

# Right shift — the extra bits roll off and are forgotten:
print(f"(x >> 3) << 3 =    {(x>>3)<<3:04b}")

# Left shift — bits are remembered; the word length grows
# to keep the most significant bit:
print(f"(x << 10) =        {x<<10:b}")
print(f"(x << 10) >> 10 = {(x<<10)>>10:b}")
```

OUTPUT

```
x =                0101
(x >> 3) << 3 =    0000
(x << 10) =        1010000000000
(x << 10) >> 10 = 101
```

- **Right shifts lose information**

  Shift out three bits of a four-bit word and they are gone. Shifting back does not bring them back.

- **Left shifts do not**

  There is no fixed width to overflow, so the value keeps growing. On a microcontroller register this is emphatically not true — mask the result.

- **This is where masks come in**

  If you mean an 8-bit word, you have to say so, every time. The next slides show how.

Two's Complement

Negative numbers are represented in binary by inverting all the bits and adding one.

```
comp(x) = -(x + 1)

Example: find -6, the two's complement of 6

     6    0b0110
    -6    0b1001 + 0b0001 = 0b1010
```

The leading 1 is what marks the value as negative.

ONE'S COMPLEMENT

```
Invert every bit.

Two representations of zero:
  00000000  positive zero (+0)
  11111111  negative zero (-0)
```

TWO'S COMPLEMENT

```
Invert every bit, then add 1.

One representation of zero:
  11111111 + 1 = 100000000
  -> 9 bits; the leftmost one
     overflows away, leaving
     00000000
```

- **Why two's complement wins**

  Only one zero, and ordinary binary addition gives the right answer for signed values with no special cases.

- **One's complement still has a job**

  It is what the ~ operator computes — pure bit inversion, which is what you want when comparing binary words.

The ~ Trap

~ flips every bit — including the sign bit — so the result displays as a negative number.

THE PROBLEM

```
x = 0b1010
print(~x)          # -11
print(bin(~x))     # -0b1011

# and for any x at all:
print(x | ~x)      # -1
```

A binary literal in Python is always positive — an extra hidden bit carries the sign. ~ toggles that bit too, so the value is interpreted as negative in two's complement.

THE FIX: MASK OFF THE LEADING BITS

```
x = int('1011', 2)
print(bin(x))              # 0b1011
print(bin(~x))             # -0b1100    (bad)
print(bin(~x & 0b1111))    # 0b100      (good)

x    = 0b11110000
mask = 0b11111111
print(f"~x & mask = {~x & mask:08b}")    # 00001111
```

Build a mask of all ones, the width of the word you mean, and AND it with the result. The leading ones that made the value negative are cut off.

Whenever you use ~ on a fixed-width value, mask the result. This is not optional — it is how you tell Python the word length it has no way of knowing.

Bit Fields

Pack several independent on/off states into the bits of one integer — exactly what a hardware register is.

```
# Define the bit positions:
A = int('001', 2)
B = int('010', 2)
C = int('100', 2)

# Set the initial state — A on, B on, C on:
state = A | B | C      # 111

# Check whether C is on:
if state & C:          # 111 & 100 = 100, which is nonzero
    print('C is on')

# Turn A off:
state &= ~A            # 111 & ~001 = 111 & 110 = 110

# Toggle B:
state ^= B             # 110 ^ 010 = 100

# Walk every bit in turn:
for i in range(3):
    print(f'State {i} is', "ON" if state & 1<<i else "OFF")
```

- **| sets a bit**

  OR with the mask. Bits not in the mask are left alone.

- **& tests a bit**

  AND with the mask gives a nonzero result exactly when that bit is set.

- **&= ~mask clears a bit**

  AND with the inverted mask. Everything else survives.

- **^= toggles a bit**

  XOR flips exactly the bits that are set in the mask.

- **1 << i is the mask for bit i**

  The standard way to build a single-bit mask from a bit number.

Code Development Process

Before you type any Python: a repeatable way to get from a problem statement to working code.

1	**Think the problem through**	Conceptualise the high-level steps that have to happen, in order.
2	**Write each step as a comment**	In a new, otherwise empty code file. This is your outline.
3	**Look for repetition**	If a step appears more than once, that step wants to be a function.
4	**Fill in the code under each comment**	When you don't remember the syntax, look it up. Nobody memorises it all.
5	**Re-think the loops**	English does not always describe the flow control you actually need. Loops are where outlines break down.
6	**Run it often**	Test each piece as you write it, and keep iterating from step 4 until it does what you wanted.
7	**Then simplify**	Once it works, go back through for what can be shortened and what should be modularised. Now you are done.

Coding Example: Hangman

A worked example, developed with the process on the previous slide.

Create a text-based game that lets the user guess the letters in an unknown word. The word is picked at random from a pre-defined list of available words. The word is displayed before each guess, with blanks for the letters not yet guessed. If the player finds every letter before six incorrect guesses, they win; otherwise they lose.

1

What does the program need to remember?

The secret word, the letters guessed so far, which positions have been revealed, and how many guesses remain.

2

What repeats?

Ask for a letter, check it, update the display. That is the main loop — everything else is setup or the ending.

3

What ends it?

Two conditions, checked every pass: no guesses left, or no blanks left.

Hangman: Setup and the Main Loop

Comments first, code underneath — the outline from the development process is still visible in the finished program.

```
import random

# Get the list of words from a file
words = []
with open('words.txt') as f:
    words = f.read().split('\n')

# Pick a random word from the list
random.seed()
i = random.randint(0, len(words) - 1)
word = words[i].strip().upper()

# Define the variables we need
n = 10                      # number of guesses allowed
guesses = []                # letters guessed so far
wordWithGuesses = []        # correct guesses within the word (initially blank)
for letter in word:
    wordWithGuesses.append('_')
```

Hangman: Taking a Guess

The top of the loop: show the word, ask for a letter, and record what it did.

```
while n > 0:                            # while guesses remain

    print(' '.join(wordWithGuesses))    # show the word so far

    letterGuess = input(
        f'You have {n} guesses left: ').upper()

    while letterGuess in guesses:       # already tried? ask again
        print(f'{letterGuess} was already guessed')
        letterGuess = input(
            f'You have {n} guesses left: ').upper()

    guesses.append(letterGuess)

    if letterGuess in word:             # hit — reveal every match
        for (idx, letter) in enumerate(word):
            if letter == letterGuess:
                wordWithGuesses[idx] = letter
    else:                               # miss — one guess gone
        n -= 1
```

- **The inner while validates the input**

  It keeps asking until the player gives a letter they have not already tried. A loop, not an if — they might repeat themselves twice.

- **A hit reveals every occurrence**

  enumerate() gives the position as well as the letter, so each matching blank in wordWithGuesses can be filled in.

- **Only a miss costs a guess**

  n comes down in the else branch alone, which is what makes the game winnable.

Hangman: Ending the Game

The bottom of the same loop: two checks, either of which finishes it.

```
    if n == 0:                      # out of guesses: player loses
        print(f'\nSorry, the word was {word}')

    if "_" not in wordWithGuesses:  # no blanks left: player wins
        print(word)
        print(f'\nYOU WIN WITH {n} MOVES LEFT!')
        n = 0                       # ends the outer while loop
```

- **Both checks run every pass**

  They are independent ifs, not a chain — the player can run out of guesses on the same turn they would have won.

- **Setting n = 0 ends the loop**

  There is no break here: zeroing the counter makes the while condition false on the next test.

- **Everything is inside the while**

  Note the indentation — these lines sit at the same level as the guessing code on the previous slide.

LAB ASSIGNMENT –OPTIONAL EXTRA CREDIT

Lab X — Mastermind

- The computer creates a random sequence of 4 values selected from 1, 2, 3, 4, 5, 6.
- The player has 12 turns to guess the correct sequence.
- If a guess is not exactly 4 numerical values in the range 1–6, prompt the player to re-enter it.
- After each guess, display the number of entries with the correct symbol AND position as a closed circle: ●
- ...and the number with the correct symbol but the wrong position as an open circle: ○

TOPICS COVERED

- Loops and break statements
- User input
- f-strings
- Basic list operations
- List comprehension with embedded loops and conditionals
- The code development process

07
objects

Object-oriented programming to structure larger code bases

Classes & Objects

A class is the cookie cutter. An object is a cookie.

CLASS

the description

- **Attributes**

  the variables every object of this class will have

- **Methods**

  the functions every object of this class can run

- **PascalCase by convention**

  MyClass

OBJECT

an instance of a class

- **Instance variables**

  each object holds its own values, independent of the others

- **Access to the class's methods**

  and to its class variables, which all instances share

- **camelCase by convention**

  myObject

Why Bother?

Four reasons the extra structure pays for itself as a program grows.

Abstraction

Bundle the data and the procedures that act on it into one thing. The structure of the code starts to match the structure of the problem.

Planning at scale

Large programs become a set of objects with clear responsibilities rather than one long file of functions and globals.

Hidden internals

A class can keep its workings private, so other code depends on what it does rather than how it does it.

Extension and reuse

Build a new class on an existing one and add to it, without touching the original — and drop the same class into the next program unchanged.

Almost Everything Is an Object in Python

Which is why the dot notation shows up on things that don't look like objects at all.

```
[0,1,2,3,2,2].count(2)       # a list literal has methods

{'a':1, 'b':2, 'c':3}.keys()    # so does a dict literal

"abcd".capitalize()          # and a string literal

(1.2).__add__(3.45)          # + is a method call

print.__doc__                # even functions have attributes
```

- **The dot means "belonging to"**

  Every method you have called so far — s.upper(), l.append(), f.read() — was a method on an object.

- **Operators are methods too**

  a + b calls a.__add__(b). Defining that method on your own class makes + work on it.

- **id() shows the identity**

  Try x = 123; id(x); id('abc'); x = 'abc'; id(x) — the id follows the object, not the name.

- **Which is what makes = a binding**

  The shallow-copy behaviour from Part 4 is a direct consequence: names refer to objects, they do not contain them.

Class Example

class opens the definition; everything indented under it belongs to the class.

```
class Student:
    'student class definition'      # optional class documentation string

    count = 0                       # class variable — shared by all instances

    # Magic methods:
    # constructor (optional, called when instantiating a new object):
    def __init__(self, first, last, grade=100):
        self.first = first          # instance variables
        self.last = last
        self.grade = grade
        Student.count += 1          # increment for each new instance

    # destructor (optional, called on deletion via 'del object'):
    def __del__(self):
        Student.count -= 1          # decrement when an instance goes away

    # Class methods:
    def displayStudent(self):
        print(f"Name: {self.first} {self.last}, Grade: {self.grade}")
```

Using the Class

Call the class like a function and you get an object back.

```
# Create objects with defined instance variables:
s1 = Student("Abbie", "Normal", 0)
s2 = Student("Sam", "Spade", 100)

# Access class methods and variables:
s1.displayStudent()
s2.displayStudent()
print(f"# of students = {Student.count}")

# Change instance variables:
s1.first = "Abby"
s1.grade = 100
s1.displayStudent()

# Delete an instance:
del s2
print(f"# after deletion = {Student.count}")
```

- **self is the object itself**

  Every method takes it as its first parameter, and Python supplies it automatically. You never pass it at the call site.

- **Instance variables are per-object**

  s1.grade and s2.grade are separate values. Changing one does not touch the other.

- **Class variables are shared**

  Student.count belongs to the class, not to any instance — which is what makes it usable as a running total.

- **Default arguments work here too**

  grade=100 in __init__ means the third argument may be left out.

Built-In Class Attributes

Every class carries these, whether you define anything or not. Access them as ClassName.__attribute__

ATTRIBUTE	WHAT IT HOLDS
`__dict__`	Dictionary containing the class namespace
`__doc__`	The class documentation string, or None if there isn't one
`__name__`	The class name
`__module__`	Module the class is defined in — "__main__", or the name of the imported module
`__bases__`	Tuple of base classes, in the order they appear in the base class list

These are how tools like help() and dir() work — and how you can inspect a class you did not write.

Class Inheritance

A child class starts with everything the parent has, then adds to it.

```
class ParentClass:
    def __init__(self):
        # instantiation code
        pass


class ChildClass(ParentClass):     # parent name as an argument
    def __init__(self):
        super().__init__()         # call the parent's init, or:
                                   # ParentClass.__init__(self)
        # instantiation code...
```

- **Everything is inherited**

  Class variables, instance variables and methods all come across without being rewritten.

- **Add whatever is specific**

  New variables and new methods can be defined on the child alone.

- **Call super().__init__()**

  The parent's constructor does not run automatically. Call it first, then do the child's own setup.

- **Redefine to override**

  A method with the same name as the parent's replaces it for objects of the child class.

Class Inheritance Example

Three levels of payroll: each class overrides calculate_payroll(), and CommissionEmployee builds on SalaryEmployee's version.

```
class Employee:
    def __init__(self, id, name):
        self.id = id
        self.name = name

class SalaryEmployee(Employee):              # inherit from Employee
    def __init__(self, id, name, weekly_salary):
        super().__init__(id, name)
        self.weekly_salary = weekly_salary
    def calculate_payroll(self):
        return self.weekly_salary

class HourlyEmployee(Employee):              # inherit from Employee
    def __init__(self, id, name, hours_worked, hour_rate):
        super().__init__(id, name)
        self.hours_worked = hours_worked
        self.hour_rate = hour_rate
    def calculate_payroll(self):
        return self.hours_worked * self.hour_rate

class CommissionEmployee(SalaryEmployee):    # inherit from SalaryEmployee
    def __init__(self, id, name, weekly_salary, commission):
        super().__init__(id, name, weekly_salary)
        self.commission = commission
    def calculate_payroll(self):
        fixed = super().calculate_payroll()  # the parent's answer...
        return fixed + self.commission       # ...plus this class's addition
```

Class Composition

Inheritance says "is a". Composition says "has a".

```
# Inheritance: "is a"
#   inherit from a base class

class SalaryEmployee(Employee):
    ...
```

```
# Composition: "has a"
#   hold a reference to another object

class MarsRover:
    self.vehicle = Vehicle(...)
```

```
class Vehicle:
    def __init__(self, num_wheels, fuel_type):
        self.num_wheels = num_wheels
        self.fuel_type = fuel_type
        self.mileage = 0
    def report_mileage(self):
        return f"The vehicle has traveled {self.mileage} miles"

class MarsRover:
    def __init__(self, name, landing_date, num_wheels, fuel_type):
        self.vehicle = Vehicle(num_wheels, fuel_type)  # extend via composition
        self.landing_date = landing_date

theRover = MarsRover("Perseverance", "02/18/2021", 6, "plutonium-238")
print(theRover.vehicle.report_mileage())
```

Public vs. Private Class Members

Prepend two underscores to a name and it becomes accessible only from inside the class.

```
class MyClass:
    __x = -1

    def myPublicMethod(self):
        print('public method called')
        self.__myPrivateMethod()

    def __myPrivateMethod(self):
        print('private method called')
        print(self.__x)


c = MyClass()           # instantiate the class
c.myPublicMethod()      # prints both lines
c.__myPrivateMethod()   # AttributeError — method is private
print(c.__x)            # AttributeError — __x is private
```

- **Private members are for the class's own use**

  Everything else forms the interface other code is allowed to depend on.

- **It is name mangling, not enforcement**

  The interpreter renames __attribute to _ClassName__attribute. Determined code can still reach it — the convention is a signpost, not a lock.

- **Inside the class, use self.__x**

  The mangled name only resolves within the class body.

Magic (Dunder) Methods

Named with double underscores at both ends. Python calls them for you, in response to ordinary syntax.

`__init__()`	Called when instantiating the class
`__iter__()`	Defines iteration behaviour — makes the object work in a for loop
`__str__()`	Dictates the output of print(myObject)
`__len__()`	Defines the result of len(myObject)
`__contains__()`	Defines the behaviour of the in operator
`__getitem__()`	Invoked when reading a value with a key: obj[key]
`__setitem__()`	Invoked when assigning with a key: obj[key] = value
`__delitem__()`	Invoked when deleting an item: del obj[key]
`__add__()`	Defines the result of the + operator
`__iadd__()`	Defines the result of the += operator

...and many more. These are what let your own classes behave like built-in types

Next Steps

ESP32 hardware and MicroPython

MicroPython is Python with a smaller standard library. Key language structure including flow control, functions, lists, dicts, classes, exceptions, etc. are unchanged.

General purpose input/output (GPIO)

Configuring and using GPIO pins to interact with the physical world.